

Examen DeliverUS - Modelo E - Control de Saturación de Pedidos

Recuerde que DeliverUS está descrito en: <https://github.com/IISII2-IS>

Enunciado del examen

Se ha de implementar la interfaz gráfica de algunos requisitos funcionales de los propietarios, en concreto relacionados con el **Control de Saturación de Pedidos** de sus restaurantes.

Se ha detectado que algunos restaurantes se saturan con demasiados pedidos simultáneos. Para solucionar esto, el sistema backend ha introducido una ventana temporal de 2 horas. Los propietarios necesitan visualizar el estado de saturación de sus restaurantes y poder marcar hasta un máximo de **3** restaurantes como “Ilimitados” para que no se vean afectados por estos límites.

Es necesaria la implementación de la parte de frontend de los siguientes requisitos funcionales:

RF.01 – Visualización de la saturación de pedidos

Como propietario,

Quiero ver en el listado de mis restaurantes el número de pedidos recibidos en las últimas 2 horas y si el restaurante se encuentra saturado,

Para conocer el estado actual de la carga de trabajo de mis restaurantes de un simple vistazo.

Pruebas de aceptación

- El listado de restaurantes se muestra en `RestaurantsScreen.js`.
 - Para cada restaurante se muestra un texto o etiqueta indicando el número de pedidos de las últimas 2 horas (`ordersInLastTwoHours`).
 - Si un restaurante está cerrado por límite de pedidos (`isClosedByLimit` es verdadero) y no es ilimitado, debe mostrarse de forma destacada el texto “Saturado”.
-

RF.02 – Conmutar el estado Ilimitado de un restaurante

Como propietario,

Quiero poder marcar o desmarcar un restaurante como ilimitado mediante un botón,

Para que no se le apliquen las restricciones de saturación.

Pruebas de aceptación

- En `RestaurantsScreen.js` (o en la tarjeta de cada restaurante), se muestra un botón para activar/desactivar el estado ilimitado (`isUnlimited`).
 - Al pulsar el botón, la vista se actualiza reflejando el nuevo estado.
 - Si se intenta establecer un cuarto restaurante como ilimitado, la acción fallará y se deberá mostrar un mensaje de error advirtiendo que se ha alcanzado el límite de 3 restaurantes ilimitados.
-

Ejercicios

1. Visualización del estado de saturación y pedidos. 4 puntos

Trabaje el RF.01 en el fichero `./DeliverUS-Frontend-Owner/src/screens/restaurants/RestaurantsScreen.js` realizando todos los cambios necesarios para mostrar la información de saturación.

Aspectos a tener en cuenta:

- Modificar la tarjeta (Card) de restaurante existente para incluir el dato de `ordersInLastTwoHours`.

- Mostrar visualmente si el restaurante está “Saturado” comprobando la propiedad `isClosedByLimit`. Puede usar un componente `TextError` o un pequeño `Badge` para que sea visible.
-

2. Botón para conmutar estado ilimitado. 3,5 puntos

Implemente la parte visual e interactiva del RF.02 en el fichero `./DeliverUS-Frontend-Owner/src/screens/restaurants/RestaurantsScreen.js`.

Aspectos a tener en cuenta:

- Añadir un botón (por ejemplo, usando un `Pressable` con un icono de `MaterialCommunityIcons` como `infinity` si es ilimitado, o `infinity-off` / un color atenuado si no lo es).
 - Al pulsar el botón, se debe llamar a la función `toggleIsUnlimited(restaurantId)` que ya se le proporciona en `./DeliverUS-Frontend-Owner/src/api/RestaurantEndpoints.js` y que realiza la petición `PATCH` necesaria al backend.
 - Tras el éxito de la petición, el listado de restaurantes debe actualizarse para reflejar los cambios.
-

3. Gestión de error por límite de restaurantes ilimitados. 2,5 puntos

Implemente el control de errores del RF.02 asegurándose de capturar correctamente el rechazo cuando se excede el límite.

Aspectos a tener en cuenta:

- Si la petición al backend devuelve un error (debido al `409 Conflict` cuando se intenta marcar un cuarto restaurante), debe capturarlo.
 - Se debe mostrar un mensaje al usuario utilizando `showMessage` de `react-native-flash-message` indicando claramente que no se pueden tener más de 3 restaurantes ilimitados.
 - El estado visual del botón en el frontend debe mantenerse consistente (es decir, no debe aparecer como “ilimitado” si el backend lo rechazó).
-

Procedimiento de entrega

1. Borrar las carpetas `DeliverUS-Backend/node_modules`, `DeliverUS-Frontend-Owner/node_modules` y `DeliverUS-Frontend-Owner/.expo`.
2. Crear un ZIP que incluya todo el proyecto. **Importante: Comprueba que el ZIP no es el mismo que te has descargado e incluye tu solución**
3. Avisa al profesor antes de entregar.
4. Cuando el profesor te dé el visto bueno, puedes subir el ZIP a la plataforma de Enseñanza Virtual. **Es muy importante esperar a que la plataforma te muestre un enlace al ZIP antes de pulsar el botón de enviar.** Se recomienda descargar ese ZIP para comprobar lo que se ha subido. Una vez realizada la comprobación, puedes enviar el examen.

Preparación del Entorno

a) Windows

- Abre una terminal y ejecuta el siguiente comando:

```
npm run install:all:win
```

b) Linux/MacOS

- Abre una terminal y ejecuta el siguiente comando:

```
npm run install:all:linux
```

Ejecución

Backend

- Para **recrear las migraciones y seeders**, abre una terminal y ejecuta el siguiente comando:

```
npm run migrate:backend
```

- Para **iniciar el backend**, abre una terminal y ejecuta el siguiente comando:

```
npm run start:backend
```

Frontend

- Para ****ejecutar la aplicación frontend del owner****, abre una nueva terminal y ejecuta el siguiente comando:

```
npm run start:frontend
```

Depuración

- Para **depurar el frontend**, asegúrate de que **SÍ** haya una instancia en ejecución del frontend que desees depurar, y usa las herramientas de depuración del navegador.